Uniwersytet Kazimierza Wielkiego | Wydział Informatyki

Inżynieria wiedzy i maszynowe uczenie · spotkanie 2

Silnik wnioskujący: w przód i wstecz

Jak program dochodzi do wniosku?

mgr inż. Marek Andryszczyk
Wydział Informatyki UKW · Informatyka, II rok · 2026/27

Cele dzisiejszego spotkania

1. **Zapisujesz wiedzę jako dane**
   reguły z Z1 przenosisz do pliku JSON

2. **Wnioskujesz ręcznie w przód i wstecz**
   karta pracy, zadania 1–2

3. **Uzupełniasz silnik w Pythonie**
   TODO 1–4 i autotest 5/5

4. **Rozumiesz konflikt reguł**
   kolejność reguł w pliku to też wiedza

Plan dzisiejszych zajęć: 90 minut

00–10	Kontrola środowiska i przegląd Z1: reguła z drzewa	organizacja
10–18	Teoria: wiedza jako dane, format bazy w JSON	teoria
18–31	Teoria: wnioskowanie w przód i wstecz · karta pracy 1–2	teoria
31–47	Twoja baza w JSON: zasady, błędy, sprawdz_format.py	ćwiczenie
47–62	Pokaz przykładu · TODO 1–2: pętla w przód · konflikt reguł	kod
62–80	TODO 3–4: pytanie i rekurencja wstecz · autotest 5/5	kod
80–87	Silnik na własnej bazie · karta pracy 3	ćwiczenie
87–90	Podsumowanie i zadanie Z2	organizacja

Od drzewa z Z1 do działającego programu



SPOTKANIE 2

Dziś

JSON z Twoimi regułami, silnik z TODO 1–4, autotest 5/5, pierwsza konsultacja na własnej bazie.

SPOTKANIE 3

Za tydzień

Wyjaśnienia „jak?" i „dlaczego?", weryfikacja bazy i interfejs w Streamlit: kamień milowy A.

01

Wiedza zapisana jako dane

Od if/elif do reguł w pliku JSON · ok. 8 min

Uniwersytet Kazimierza Wielkiego
Wydział Informatyki

Doradca kawowy zapisany jako if/elif

se/kawa_ifelse.py (fragment)

```
if temperatura > 22:
    if kofeina_ok == "nie" or pora_dnia == "wieczór":
        kawa = "mrożona kawa bezkofeinowa"
    elif dodatek == "czekolada":
        kawa = "chocolate cold coffee"
    elif dodatek == "mleko":
        kawa = "iced latte"
    else:
        kawa = "cold brew"
else:
    if kofeina_ok == "nie" or pora_dnia == "wieczór":
        kawa = "latte bezkofeinowe"
    elif dodatek == "czekolada":
        kawa = "mocha"
    …  # jeszcze 10 linii
```

Rozbudowa

Nowy dodatek „syrop” to zmiany w dwóch gałęziach i w pytaniu.

Sprawdzenie

Ekspert barista nie przeczyta kodu i nie poprawi wiedzy.

Wyjaśnienie

Program nie umie powiedzieć, dlaczego polecił tę kawę.

Ta sama wiedza jako lista reguł

se/kawa_reguly.py (fragment)

```
REGULY = [
  {"jesli": {"rodzaj": "zimna", "dodatek": "mleko"},
   "to": "iced latte"},
  {"jesli": {"rodzaj": "gorąca", "dodatek": "czekolada"},
   "to": "mocha"},
  …
]
```

```
def dopasuj(fakty):
    for regula in REGULY:
        if all(fakty.get(a) == w for a, w in regula["jesli"].items()):
            return regula["to"]
```

Wiedza oddzielona od mechanizmu

Funkcja dopasuj nie zawiera słowa „kawa”. Ta sama funkcja zadziała dla roweru, pralki czy rośliny.

Czego brakuje?

porównań > i <=

wniosków pośrednich (łańcuchów)

opisów dla użytkownika

Format bazy wiedzy w JSON

wiedza/kawa.json (fragment)

```
{
 "nazwa": "Dobór kawy",
 "cel": "kawa",
 "atrybuty": {
  "temperatura": {"typ": "liczba",
     "pytanie": "Jaka jest temperatura (°C)?",
     "zakres": [-30, 45]},
  "dodatek": {"typ": "wybor", "pytanie": "…",
     "wartosci": ["mleko", "czekolada", "brak"]},
  "rodzaj_kawy": {"typ": "wniosek", …}
 },
 "reguly": [
  {"id": "R1", "jesli": [["temperatura", ">", 22]],
   "to": ["rodzaj_kawy", "zimna"],
   "metafora": "Jest ciepło, więc kawa na zimno."}
 ]
}
```

Pole	Znaczenie
cel	atrybut decyzji końcowej
atrybuty	typ, pytanie, wartości lub zakres
jesli	warunki [atrybut, op, wartość], łączone ORAZ
to	wniosek [atrybut, wartość]
metafora	dlaczego reguła ma sens
coto	opis decyzji końcowej



Trzy typy atrybutów i trzy poziomy łańcucha

`"typ": "liczba"`

temperatura

pyta użytkownika, podaje zakres

`"typ": "wybor"`

dodatek, pora_dnia

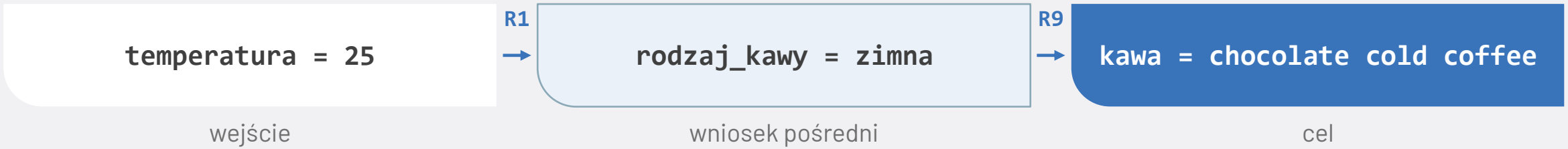
pyta, podaje listę wartości

`"typ": "wniosek"`

rodzaj_kawy, moc, kawa

ustalają reguły, nigdy nie pytamy

Łańcuch trzech poziomów atrybutów = co najmniej dwa odpalenia reguł



Uwaga: „3 poziomy" to nie „3 reguły". Liczymy poziomy atrybutów: dane wejściowe, wniosek pośredni, decyzja.

02

Wnioskowanie w przód

Od faktów do wniosku · karta pracy, zadanie 1

Uniwersytet Kazimierza Wielkiego
Wydział Informatyki

Baza faktów i dopasowanie warunku

se/silnik.py: gotowe

```
OPERATORY = {
    "==": operator.eq, "!=": operator.ne,
    ">": operator.gt,  ">=": operator.ge,
    "<": operator.lt,  "<=": operator.le}

def warunek_spelniony(warunek, fakty):
    atrybut, op, wartosc = warunek
    return (atrybut in fakty and fakty[atrybut] is not None
            and OPERATORY[op](fakty[atrybut], wartosc))
```

```
fakty = {"temperatura": 18, "pora_dnia": "rano"}
warunek_spelniony(["temperatura", ">", 22], fakty)  # False
```

TODO 1: regula_spelniona

True, gdy WSZYSTKIE warunki z regula["jesli"] są spełnione. Podpowiedź: all(...).

Nieznane ≠ fałszywe

Brak atrybutu w faktach oznacza, że reguła jeszcze nie może odpalić. Może odpalić później, gdy fakt się pojawi.

Wnioskowanie w przód: od faktów do wniosku




```
powtarzaj:
    dla każdej reguły po kolei:
        jeśli wniosek nieznany i reguła spełniona:
            dodaj wniosek do faktów, zapisz id w śladzie
aż do przebiegu bez zmian
```

Kiedy w przód?

Wszystkie dane znamy od razu (formularz, czujniki). Wada: odpala też reguły zbędne dla celu.

Przykład: w przód krok po kroku

Przebieg	Reguła	Warunki	Nowy fakt
1	R2	temperatura <= 22	rodzaj_kawy = gorąca
1	R4	kofeina_ok = tak ORAZ rano	moc = mocna
1	R15	gorąca ORAZ brak ORAZ mocna	kawa = espresso
2	—	żadna reguła nie wnosi nic nowego	koniec pętli

Fakty początkowe: temperatura = 18, pora_dnia = rano, dodatek = brak, kofeina_ok = tak

Wynik

espresso

ślad: R2 → R4 → R15

Po co przebieg 2?

Dopiero przebieg bez zmian dowodzi, że z faktów nic więcej nie wynika.

03

Wnioskowanie wstecz

Od celu do pytań · karta pracy, zadanie 2

Uniwersytet Kazimierza Wielkiego | Wydział Informatyki

Wnioskowanie wstecz: od celu do pytań

Cel

`kawa = ?`

Szukamy reguł, których wnioskiem jest kawa: R7, R8, ...

→

Podcele

`rodzaj_kawy, moc`

Warunek reguły dotyczy wniosku? Ustalamy go rekurencyjnie.

→

Pytanie

`temperatura?`

Atrybut wejściowy bez reguł: pytamy użytkownika i zapamiętujemy.

→

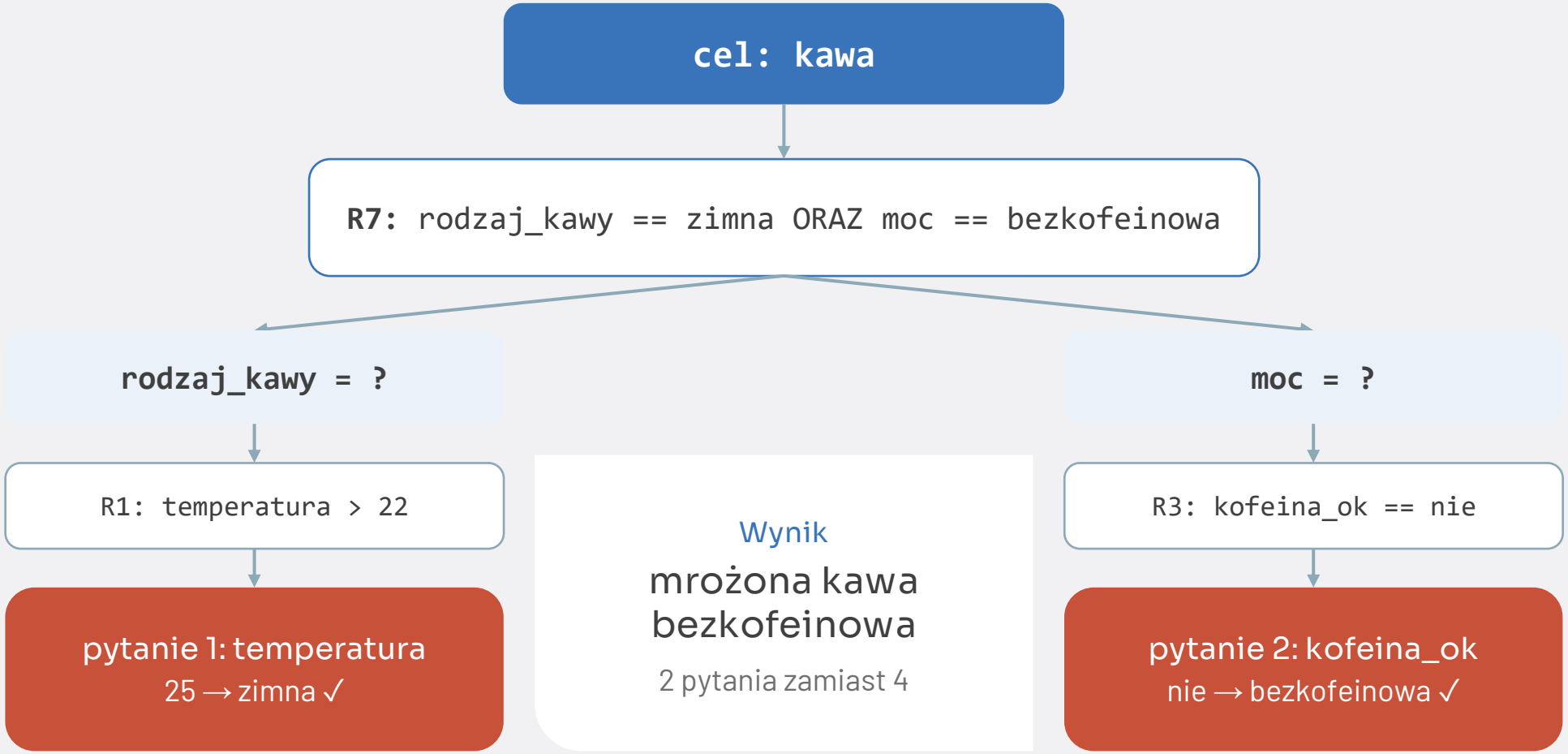
Decyzja

`pierwsza spełniona reguła`

Niespełniony warunek: reguła odpada, sprawdzamy następną.

Kiedy wstecz? W dialogu z użytkownikiem i gdy zdobycie danej kosztuje (badanie, pomiar, czas). System pyta tylko o to, co jest potrzebne do sprawdzanej hipotezy.

Drzewo celów: tylko dwa pytania



W przód czy wstecz?

	Wnioskowanie w przód	Wnioskowanie wstecz
Sterowane	danymi	celem (hipotezą)
Punkt startowy	znane fakty	atrybut celu, np. kawa
Pytania do użytkownika	wszystkie dane z góry	tylko o dane potrzebne dla sprawdzanej reguły
Wynik	wszystkie wnioski, które wynikają z faktów	wartość jednego celu i podcele po drodze
Zastosowania	formularz, monitoring, konfiguratory	diagnostyka, doradztwo w dialogu
U nas	formularz Streamlit (spotkanie 3)	konsultacja w konsoli

Test spójności: dla tych samych danych oba tryby powinny dać ten sam wynik. Różnica sygnalizuje konflikt lub lukę.

04

Twoja baza wiedzy w JSON

Jak napisać plik, który silnik zrozumie · ok. 16 min

Uniwersytet Kazimierza Wielkiego
Wydział Informatyki

Szkielet pliku: cztery klucze i kolejność pracy

wiedza/moja_dziedzina.json: szkielet

```
{
 "nazwa": "Dobór kawy",          ← tekst dla człowieka
 "cel": "kawa",                  ← nazwa atrybutu decyzji
 "atrybuty": {                   ← SŁOWNIK: nazwa → opis
   "temperatura": {…},
   "rodzaj_kawy": {…},
   "kawa": {…}
 },
 "reguly": [                     ← LISTA: kolejność ważna
   {"id": "R1", …},
   {"id": "R2", …}
 ]
}
```

strzałki to opis na slajdzie: JSON nie dopuszcza komentarzy

KOLEJNOŚĆ PRACY

1. **nazwa i cel**
   co system ma ustalić?
2. **dane wejściowe**
   liczba lub wybor: o co zapytamy?
3. **wnioski**
   pośrednie i cel, z listą wartości
4. **reguły pośrednie**
   ustalają wnioski pośrednie
5. **reguły decyzji**
   ustalają cel, mają coto
6. **sprawdzenie**
   python se/sprawdz_format.py

Deklaracja atrybutu: co musi mieć każdy typ

"liczba" pyta, podaje zakres

```
"temperatura": {
  "typ": "liczba",
  "pytanie": "Temperatura (°C)?",
  "zakres": [-30, 45]
}
```

typ + pytanie + zakres

"wybor" pyta, podaje listę

```
"dodatek": {
  "typ": "wybor",
  "pytanie": "Jaki dodatek?",
  "wartosci": ["mleko",
     "czekolada", "brak"]
}
```

typ + pytanie + wartosci

"wniosek" ustalają reguły

```
"rodzaj_kawy": {
  "typ": "wniosek",
  "wartosci": ["zimna",
               "gorąca"]
}
```

typ + wartosci, bez pytania

Nazwy atrybutów

`pora_dnia, kofeina_ok`

małe litery, _ zamiast spacji, bez ogonków

Nazwy pól formatu

`wartosci, wybor, jesli`

nie: wartości, wybór, jeśli

Wartości

`"gorąca", "wieczór"`

małe litery; polskie znaki wolno (UTF-8)

Reguła: pięć pól i zasady operatorów

reguła z decyzją końcową

```
{"id": "R9",
 "jesli": [["rodzaj_kawy", "==", "zimna"],
           ["dodatek", "==", "czekolada"]],
 "to": ["kawa", "chocolate cold coffee"],
 "metafora": "Na zimno i z czekoladą.",
 "coto": "Schłodzona kawa z czekoladą."}
```

jeden warunek też w liście:

```
"jesli": [["temperatura", ">", 22]]
        ↑↑ podwójny nawias
```

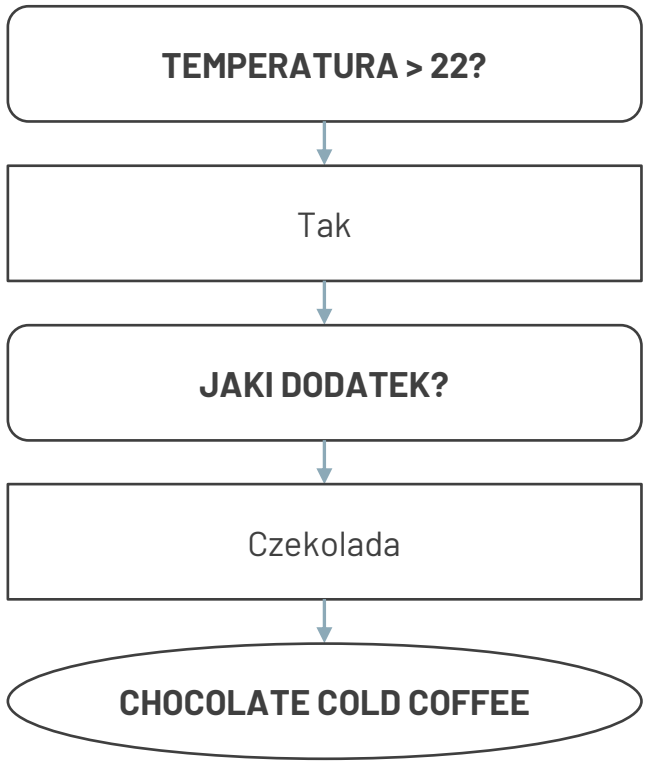
Pole	Zasada
id	unikalny tekst: "R1", "R2", …
jesli	lista trójek [atrybut, operator, wartość], łączone ORAZ
to	jedna para [atrybut, wartość]; atrybut typu wniosek
metafora	w każdej regule: dlaczego ma sens
coto	w regułach z decyzją końcową

`==` `!=` wybor, wniosek · `>` `>=` `<` `<=` liczba

LUB = dwie reguły z tym samym wnioskiem

Ścieżka drzewa staje się łańcuchem reguł

W DRZEWIE (Z1)



wniosek pośredni

```
{"id": "R1",
 "jesli": [["temperatura", ">", 22]],
 "to": ["rodzaj_kawy", "zimna"]}
```

decyzja końcowa

```
{"id": "R9",
 "jesli": [["rodzaj_kawy", "==", "zimna"],
           ["dodatek", "==", "czekolada"]],
 "to": ["kawa", "chocolate cold coffee"]}
```

Sześć najczęstszych błędów w pliku JSON

SKŁADNIA I PISOWNIA

Przecinek na końcu listy

```
["mleko", "brak",]
```

→ usuń ostatni przecinek

Liczba jako tekst

```
["temperatura", ">", "22"]
```

→ liczby bez cudzysłowu: 22

Inna pisownia wartości

```
"Zimna" zamiast "zimna"
```

→ kopiuj wartości z deklaracji

STRUKTURA: JSON POPRAWNY, A SILNIK NIE DZIAŁA

Jeden warunek bez listy

```
"jesli": ["moc", "==", "mocna"]
```

→ "jesli": [["moc", "==", "mocna"]]

Polskie znaki w polach

```
"jeśli", "wartości", "wybór"
```

→ jesli, wartosci, wybor

Porównanie tekstu

```
["pora_dnia", ">", "rano"]
```

→ dla tekstu tylko == i !=

```
python se/sprawdz_format.py wiedza/moja_dziedzina.json
[BŁĄD] Składnia JSON, linia 5, kolumna 59: Illegal trailing comma before end of array
```




Lista kontrolna i sprawdz_format.py

LISTA KONTROLNA

- ✓ plik w wiedza/, kodowanie UTF-8, jeden obiekt {...}
- ✓ cel zadeklarowany z typem wniosek
- ✓ wejścia: typ, pytanie, wartosci albo zakres
- ✓ każda reguła: id, jesli [[...]], to [atrybut, wartość]
- ✓ metafora w każdej regule, coto przy decyzji
- ✓ nazwy kopiowane z deklaracji, bez literówek

terminal VS Code: wynik dla przykładowej bazy roweru

```
python se/sprawdz_format.py wiedza/moja_dziedzina.json

[BŁĄD] reguła R2: pojedynczy warunek też musi
       być w liście: "jesli": [["dystans", ">=", 50]]
[UWAGA] reguła R3: brak 'metafora'

Format — błędy: 1, uwagi: 1.

Wymagania Z2:
  [NIE] co najmniej 20 reguł (jest 3)
  [TAK] łańcuch co najmniej 3 poziomów atrybutów
  [NIE] pole 'metafora' w każdej regule
  [TAK] pole 'coto' w każdej regule z decyzją
```

05

Silnik w Pythonie

TODO 1–4, autotest i Twoja baza · ok. 40 min

Uniwersytet Kazimierza Wielkiego
Wydział Informatyki

Pokaz: kompletny przykład w działaniu

przyklad_kompletny/

wiedza/kawa.json baza (16 reguł)

se/silnik.py TODO 1–4 rozwiązane

se/test_silnik.py autotest 5/5

se/pokaz.py krok po kroku

se/test_kawa_z2.py wzór testów Z2

se/sprawdz_format.py kontrola formatu

README.md jak uruchomić

terminal, folder przyklad_kompletny (wynik skrócony)

```
python se/pokaz.py 3
b) 25°C, kofeina nie:
  pytanie 1: Jaka jest temperatura (°C)? → 25
    (bo sprawdzam R1: temperatura > 22 …)
  pytanie 2: Czy możesz pić kofeinę? → nie
    (bo sprawdzam R3: kofeina_ok == 'nie' …)
Wynik: mrożona kawa bezkofeinowa | pytań: 2
       ślad: R1 -> R3 -> R7
Nie zapytał o: pora_dnia, dodatek
```

Tak ma działać Twój silnik po TODO 1–4: najpierw na kawie, potem na Twojej bazie.

Dzisiejsza praca: cztery luki i autotest

TODO 1
regula_spelniona
all() po warunkach

→

TODO 2
pętla w przód
dopisz fakt, ślad, zmiana

→

TODO 3
pytanie użytkownika
zapisz odpowiedź w faktach

→

TODO 4
rekurencja wstecz
ta sama funkcja dla podcelu

terminal VS Code, katalog główny repozytorium

```
python se/silnik.py
=== Wnioskowanie w przód ===
Wynik: espresso | reguły: R2 -> R4 -> R15

python se/test_silnik.py
Zaliczone przypadki: 5/5
```

Potem: Twoja baza

Podmień plik bazy na moja_dziedzina.json. Znajdź dane, dla których system nie daje odpowiedzi (karta pracy, zad. 5).

Pętla wnioskowania w przód w Pythonie

se/silnik.py: TODO 2 po uzupełnieniu

```
def wnioskuj_w_przod(baza, fakty):
    fakty = dict(fakty)
    slad = []
    zmiana = True
    while zmiana:
        zmiana = False
        for regula in baza["reguly"]:
            atrybut, wartosc = regula["to"]
            if (atrybut not in fakty
                    and regula_spelniona(regula, fakty)):
                fakty[atrybut] = wartosc      # TODO 2
                slad.append(regula["id"])
                zmiana = True
    return fakty, slad
```

not in fakty

fakt raz ustalony nie jest nadpisywany

zmiana = True

nowy fakt wymusza kolejny przegląd

slad

lista id reguł: materiał na wyjaśnienie „jak?”

Konflikt reguł: kolejność to też wiedza

Dane: 25°C, wieczór, czekolada, kofeina tak → rodzaj_kawy = zimna (R1), moc = bezkofeinowa (R6)

R7

`zimna ORAZ bezkofeinowa`

→ **mrożona kawa bezkofeinowa**

R9

`zimna ORAZ czekolada`

→ **chocolate cold coffee**

Pierwsza wygrywa. R7 stoi przed R9, więc wieczorem dostajemy kawę bezkofeinową. Po przeniesieniu R7 na koniec wynik zmieni się na chocolate cold coffee.

Nasza strategia: kolejność reguł

Pierwsza pasująca reguła wygrywa, bo silnik nie nadpisuje ustalonego faktu (atrybut not in fakty).

W Twojej bazie

Ważniejszą regułę postaw wyżej. W README napisz jednym zdaniem, którą i dlaczego.

Wnioskowanie wstecz w Pythonie: rekurencja

se/silnik.py: wnioskuj_wstecz (skrót)

```
reguly = [r for r in baza["reguly"] if r["to"][0] == cel]
if not reguly and typ(cel) == "wniosek":  # luka
    return None
if not reguly:                          # TODO 3
    fakty[cel] = zapytaj(cel, powod)
    return fakty[cel]
for regula in reguly:
    spelniona = True
    for atrybut, op, wartosc in regula["jesli"]:  # TODO 4
        znana = wnioskuj_wstecz(baza, atrybut, fakty,
                                zapytaj, slad, regula)
        if znana is None or not OPERATORY[op](znana, wartosc):
            spelniona = False; break
    if spelniona: ustal cel, dopisz id do śladu, return
```

TODO 3

pytamy tylko o atrybuty wejściowe i zapamiętujemy odpowiedź

TODO 4

rekurencja po warunkach, break przy pierwszym niespełnionym

Gotowe

wniosek bez reguł to luka: zwracamy None, nie pytamy



Twoja baza w silniku: cztery sprawdzenia

se/silnik.py: jedyne miejsce do zmiany

```
if __name__ == "__main__":
    PLIK_BAZY = Path(__file__).parent.parent \
        / "wiedza" / "moja_dziedzina.json"
    FAKTY = {"teren": "las",
             "dystans": 80}       # Twoje dane
    baza = wczytaj_baze(PLIK_BAZY)
    cel = baza["cel"]             # cel z pliku
```

najpierw format, potem silnik

```
python se/sprawdz_format.py wiedza/moja_dziedzina.json
python se/silnik.py
```

CZTERY SPRAWDZENIA

1. **w przód dochodzi do celu**
   dla co najmniej 5 różnych zestawów danych
2. **wstecz pyta sensownie**
   o potrzebne dane, w sensownej kolejności
3. **oba tryby zgodne**
   różnica = konflikt albo luka
4. **dane bez odpowiedzi**
   zanotuj: karta pracy, zad. 5



Z2: silnik na Twojej bazie wiedzy

Zadanie Z2 · termin: spotkanie 3 · 10 pkt

1. se/silnik.py uzupełniony: autotest 5/5
2. wiedza/moja_dziedzina.json: min. 20 reguł, łańcuch 3 poziomów
3. se/test_moja_dziedzina.py: min. 5 przypadków, w tym granica przedziału, brak wyniku, konflikt
4. README: konsultacja wstecz z komentarzem i opis strategii konfliktów

Kryterium	pkt
silnik: autotest 5/5	4
baza: 20 reguł, 3 poziomy, oba tryby	3
własne testy (min. 5)	2
README: konsultacja i strategia	1
Razem	10

Przed oddaniem: `python se/sprawdz_format.py` → 0 błędów i same TAK w części Wymagania Z2. Wzór testów: przyklad_kompletny/se/test_kawa_z2.py.

Co zabierasz z dzisiejszych zajęć

01 Wiedza w JSON jest oddzielona od silnika: ten sam kod działa dla każdej dziedziny.

02 W przód: fakty uruchamiają reguły, aż nic nowego nie wynika.

03 Wstecz: cel wyznacza podcele, system pyta tylko o potrzebne dane.

04 Konflikty rozstrzyga kolejność reguł: to też część wiedzy.

SPOTKANIE 3 · KAMIEŃ MILOWY A

Wyjaśnienia „jak?", „dlaczego?" i „co to?", weryfikacja bazy wiedzy (odpowiednik programu Cake) i interfejs w Streamlit na Twojej bazie.

Bibliografia

[1] A. Wakulicz-Deja, A. Nowak-Brzezińska, M. Przybyła-Kasperek, R. Simiński, Systemy ekspertowe. Warszawa: EXIT, 2018 (rozdz. o metodach wnioskowania).

[2] W. Traczyk, Inżynieria wiedzy. Warszawa: EXIT, 2010.

[3] Python Software Foundation, „Data Structures: Dictionaries". [Online]. Dostępne: https://docs.python.org/3/tutorial/datastructures.html

[4] Python Software Foundation, „operator: Standard operators as functions" oraz „json: JSON encoder and decoder". [Online]. Dostępne: https://docs.python.org/3/library/

[5] Ecma International, ECMA-404: The JSON Data Interchange Syntax, wyd. 2, 2017. [Online]. Dostępne: https://ecma-international.org/publications-and-standards/standards/ecma-404/

[6] Materiały kursu IWiMU 2026/27: wiedza/kawa.json, se/silnik.py, se/test_silnik.py, se/sprawdz_format.py, przyklad_kompletny/.

Uniwersytet Kazimierza Wielkiego | Wydział Informatyki

Dziękuję za uwagę



mgr inż. Marek Andryszczyk

Asystent badawczo-dydaktyczny
Wydział Informatyki UKW

marek.andryszczyk@ukw.edu.pl
ORCID: 0000-0003-2732-8631
andryszczyk.eu